arXiv:2607.12696v1 [cs.CL] 14 Jul 2026

Less Experts, Faster Decoding: Cost-Aware Speculative Decoding for Mixture-of-Experts

Jincheng Xie [*1‡] **Runheng Liu** [*2‡] **Heyan Huang** [2] **Yawen Ling** [3] **Hanbin Dai** [3] **Yu Zheng** [4 5] **Wen Hu** [3]

Abstract

Sparse Mixture-of-Experts (MoE) models have become an important approach for scaling Large Language Models (LLMs), but their inference efficiency depends strongly on expert activation patterns. Speculative decoding (SD) accelerates autoregressive generation by verifying multiple draft tokens in parallel, yet existing draft selection strategies primarily optimize acceptance likelihood. In large-scale MoE models, however, selecting draft tokens also determines the union of experts activated during verification. We observe that confidence-driven SD can introduce *expert scattering*: high-probability draft tokens may route to disjoint experts, increasing expert-weight memory traffic and reducing the speedup from speculation. Motivated by this observation, we revisit draft-tree selection under the non-uniform memory-cost structure of MoE inference. We propose EcoSpec, a cost-aware speculative decoding framework that incorporates predicted marginal expert activation cost into draft selection. With a lightweight expert predictor and a dynamic expert buffer, EcoSpec favors draft paths that preserve high acceptance likelihood while reusing experts already covered by the current verification set, without modifying the target-model verification rule. We evaluate EcoSpec on three large-scale MoE models, including DeepSeek-V3.1 (671B), Qwen3-235B-A22B, and GPT-OSS-120B, across reasoning, coding, question-answering, and dialogue benchmarks. EcoSpec consistently reduces active expert footprints and improves end-to-end decoding speed, achieving up to $1.62\times$ speedup. These results show that accounting for expert activation cost is important for efficient speculative decoding in large-scale MoE models.

1. Introduction

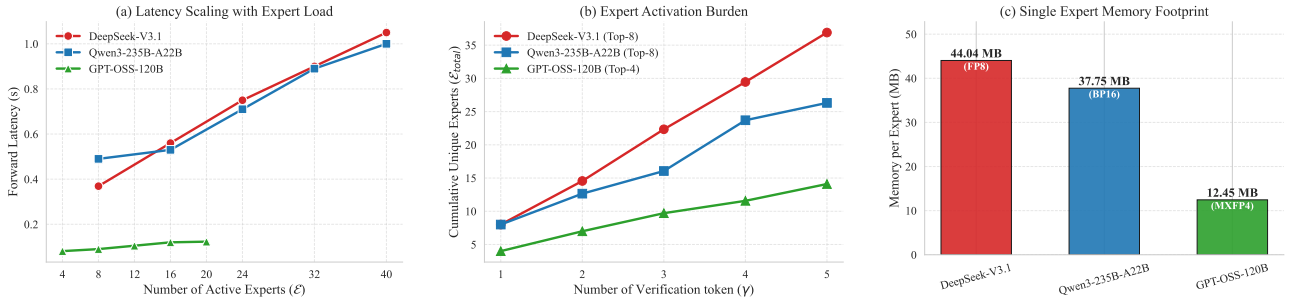
Large Language Models (LLMs) have demonstrated remarkable capabilities across logic, coding, and creative tasks (Brown et al., 2020; OpenAI et al., 2024; Grattafiori et al., 2024), yet their deployment is increasingly constrained by high inference latency and serving cost (Rajbhandari et al., 2020). The standard autoregressive decoding process generates tokens sequentially, where each step requires a full pass through the target model (Vaswani et al., 2017). As models scale to hundreds of billions of parameters, this serial dependency makes inference heavily memory-bound: the arithmetic intensity is low, and throughput is often limited by the bandwidth required to load model weights from High Bandwidth Memory (HBM) to on-chip compute units (Williams et al., 2009; Shazeer, 2019; Dao et al., 2022). Consequently, reducing decoding latency and memory traffic has become a critical priority for both real-time user experience and infrastructure efficiency.

To mitigate the serial decoding bottleneck, Speculative Decoding (SD) has been widely studied as an inference acceleration paradigm (Leviathan et al., 2023; Chen et al., 2023; Li et al., 2024a;b; 2026; Cai et al., 2024). SD uses a low-cost draft mechanism to propose multiple candidate tokens, which are then verified in parallel by the target model. When the drafts are accepted, a single target-model forward pass can advance generation by multiple tokens, thereby amortizing parameter loading and improving hardware utilization. For dense Transformers, where the same weight matrices are reused across all verified positions, this parallel verification can substantially increase arithmetic intensity and reduce the effective cost per generated token (Miao et al., 2024).

However, the dense-model assumption behind this amortization does not directly extend to sparse Mixture-of-Experts (MoE) architectures (Shazeer et al., 2017; Fedus et al., 2022), which are increasingly adopted in large-scale lan-

[*]Equal contribution [‡]Work done while interning at JDT AI Infra. [1]Department of Mathematical Sciences, Tsinghua University, Beijing, China [2]School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China [3]JDT AI Infra [4]School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu, Sichuan, China [5]School of Cyber Engineering, Xidian University, Xi'an, Shaanxi, China. Correspondence to: Wen Hu <huwen.31@jd.com>.

Preprint. July 15, 2026.



(a) **Latency vs. Expert Load** *(b)* **Expert Activation Burden** *(c)* **Single Expert Cost**

Figure 1. **The Bandwidth Bottleneck in MoE Speculative Decoding.** (a) Verification latency scales linearly with the number of active experts ($\mathcal{E}$), creating a strict latency penalty for retrieving extra experts. (b) Top-K means K activated experts in each layer. As the verification budget γ increases, standard baselines (e.g., Eagle) rapidly activate disjoint sets of experts, quickly saturating the memory bandwidth. (c) The physical memory footprint of a *single* expert is substantial. For DeepSeek-V3.1 (FP8), loading just one expert consumes **44.04 MB** of HBM bandwidth, implying that every mispredicted expert path incurs a massive I/O overhead.

guage models (OpenAI et al., 2025; DeepSeek-AI et al., 2025; Yang et al., 2025). In MoE layers, the dense feed-forward module is replaced by a pool of experts, and a router assigns each token to a small top-k subset of experts (Lepikhin et al., 2021; Du et al., 2022). Therefore, parallel verification no longer reuses a single fixed set of feed-forward weights across all verified positions. Its memory cost depends on the union of experts activated by the verified tokens: when different candidate tokens route to disjoint experts, the verifier must fetch additional expert weight blocks from HBM. As a result, verification latency in MoE speculative decoding becomes highly sensitive to expert overlap and reuse, rather than being determined mainly by the number of verified tokens.

This MoE-specific cost structure exposes a mismatch between the acceptance-driven selection objective of existing SD methods and the memory cost of MoE verification. Many recent SD methods organize draft candidates as a tree and select a verification subset primarily according to confidence or acceptance likelihood. This criterion is effective for dense models, where the main objective is to maximize the number of accepted tokens per target-model pass. In MoE verification, however, a high-probability candidate can still activate experts that are disjoint from those used by other verified tokens. Adding such a candidate may therefore enlarge the per-step expert union, increase expert-weight memory traffic, and reduce cache reuse (Huang et al., 2026; Xue et al., 2024). We refer to this expansion of the per-step expert footprint as *expert scattering*. As shown in Fig. 1, verification latency increases with the active expert footprint, while confidence-driven selection can rapidly increase the number of unique experts touched within one verification step. Consequently, expanding the verification set may erode end-to-end speedup even when acceptance rates remain comparable. This observation motivates a cost-aware draft selection objective that balances acceptance likelihood with the marginal cost of introducing new experts.

To address this issue, we propose EcoSpec, a cost-aware speculative decoding framework for MoE models. EcoSpec operates at the draft-tree selection stage, where it selects candidates for parallel verification under an acceptance–cost trade-off. Instead of ranking draft tokens primarily by acceptance likelihood, EcoSpec also accounts for the marginal expert cost induced by each candidate. This allows the selection procedure to prefer draft paths that maintain high acceptance probability while reusing experts already covered by the current verification set. Importantly, EcoSpec does not modify the target-model verification rule and therefore preserves the lossless semantics of standard speculative decoding (Leviathan et al., 2023). By aligning draft selection with the memory-cost structure of MoE inference, EcoSpec reduces unnecessary expert-weight traffic during verification. Empirically, EcoSpec achieves consistent speedups across multiple production-scale MoEs, including up to **1.62**× on Qwen3-235B, **1.50**× on GPT-OSS-120B, and **1.47**× on DeepSeek-V3.1.

Our contributions are summarized as follows:

- We identify and analyze *expert scattering* in large-scale MoE speculative decoding: confidence-driven draft selection can expand the per-step union of activated experts, increasing expert-weight memory traffic during verification.
- We propose EcoSpec, a cost-aware speculative decoding framework that incorporates marginal expert activation cost into draft-tree selection. With a lightweight expert predictor and a dynamic expert buffer, EcoSpec favors draft paths that preserve high acceptance likelihood while improving expert reuse, without changing the standard lossless verification procedure.

- We evaluate EcoSpec on three large-scale MoE models—DeepSeek-V3.1 (671B), Qwen3-235B-A22B, and GPT-OSS-120B—across diverse reasoning, coding, and dialogue benchmarks. EcoSpec consistently reduces activated experts and improves end-to-end decoding speed, achieving up to $1.62\times$ speedup over existing SD baselines.

2. Related Work

Speculative Decoding and Multi-Token Prediction. Speculative decoding (SD) accelerates LLM inference by using a lower-cost draft mechanism to propose multiple tokens, which are then verified in parallel by the target model (Leviathan et al., 2023; Chen et al., 2023). Early approaches commonly use a separate draft model to generate candidate continuations (Miao et al., 2024). Recent methods further improve draft efficiency and acceptance by adopting tree-structured drafting or auxiliary prediction heads, including Medusa and the EAGLE series (Cai et al., 2024; Li et al., 2024a;b; 2026). In parallel, Multi-Token Prediction (MTP) introduces an auxiliary training objective that enables in-model token predictors to serve as draft heads for speculative verification (DeepSeek-AI et al., 2025; Gloeckle et al., 2024). Despite architectural differences in how drafts are produced, these approaches typically prioritize draft tokens based on confidence (acceptance likelihood) and do not explicitly account for the hardware cost of activating additional experts in MoE verification. As a result, when applied to MoE models, high-confidence drafts may still trigger a rapidly expanding union of activated experts, corresponding to our Expert Scattering phenomenon and the associated memory inefficiency. EcoSpec makes this explicit by optimizing a cost-aware objective that balances acceptance likelihood against the incremental cost of activating new experts.

Efficient Inference for Mixture-of-Experts. Mixture-of-Experts (MoE) models increase model capacity while reducing per-token FLOPs, but their inference efficiency is often limited by expert-weight memory traffic and token–expert dispatch overheads (Shazeer et al., 2017; Liu et al., 2026). Prior work improves MoE execution under a given routing pattern through expert caching and prefetching (Xue et al., 2025; Huang et al., 2024), optimized token dispatch and fused MoE kernels (Gale et al., 2023), and routing or load-balancing strategies that reduce uneven expert utilization (Fedus et al., 2022; DeepSeek-AI et al., 2025). Recent systems further study MoE inference in the speculative-decoding setting. SP-MoE and MoE-SpeQ use speculative lookahead to support expert prefetching, offloading, and execution scheduling, aiming to hide or reduce expert-movement overhead during MoE serving (Chen et al., 2025; Wang et al., 2025). MoE-Spec instead reduces verification-time expert cost by imposing an expert budget and selecting only a subset of experts to load during speculative verification (McDanel et al., 2026). These methods either optimize runtime execution under predicted or routed expert demand, or change the verification-time expert budget. EcoSpec addresses a different stage of the decoding pipeline: it preserves the standard target-model verifier and lossless speculative decoding semantics, and only changes which draft-tree nodes are selected before verification. Therefore, EcoSpec is complementary to MoE runtime optimizations, since expert caching, prefetching, offloading, or optimized dispatch can still be applied after EcoSpec reduces the expert working set induced by the selected verification nodes.

3. MoE Speculative Decode Bottleneck

Speculative Decoding (SD) accelerates autoregressive generation by verifying multiple draft tokens in a single target-model forward pass. For dense Transformers, the memory cost of verifying γ tokens is close to that of verifying a single token, because all verified positions reuse the same dense weight matrices within the verification batch. Thus, increasing the verification budget mainly improves the amortization of target-model parameter loading and can reduce the effective cost per generated token.

This premise becomes less reliable for sparse MoE models. For a MoE layer ℓ, let $\mathcal{S}_\ell(x_t)$ denote the set of experts activated by the t-th verified token. Verifying a draft sequence $x_{1:\gamma}$ requires accessing the union of experts activated by all verified tokens at that layer:

$$\mathcal{E}^{\ell}_{\text{verify}} = \bigcup_{t=1}^{\gamma} \mathcal{S}_\ell(x_t). \tag{1}$$

The verification footprint is therefore determined by the size of this union, aggregated across MoE layers, rather than by the number of verified tokens alone. If selected draft tokens route to overlapping experts, verification can reuse expert weights. If they route to disjoint experts, the verifier must fetch additional expert weight blocks from HBM, increasing memory traffic within the same speculative step.

This creates an expert-scattering effect in MoE speculative decoding. Standard SD selection strategies are typically designed to maximize acceptance likelihood and are not aware of expert locality. As a result, adding more high-confidence draft tokens can enlarge the per-step expert union even when those tokens are likely to be accepted. In practice, $|\mathcal{E}^{\ell}_{\text{verify}}|$ can grow rapidly with the verification budget γ, reducing expert reuse and creating an additional memory-traffic bottleneck.

We profile DeepSeek-V3.1 (671B), Qwen3-235B-A22B, and GPT-OSS-120B on GSM8K (Cobbe et al., 2021) under identical execution settings. Fig. 1(a) shows that verifica-

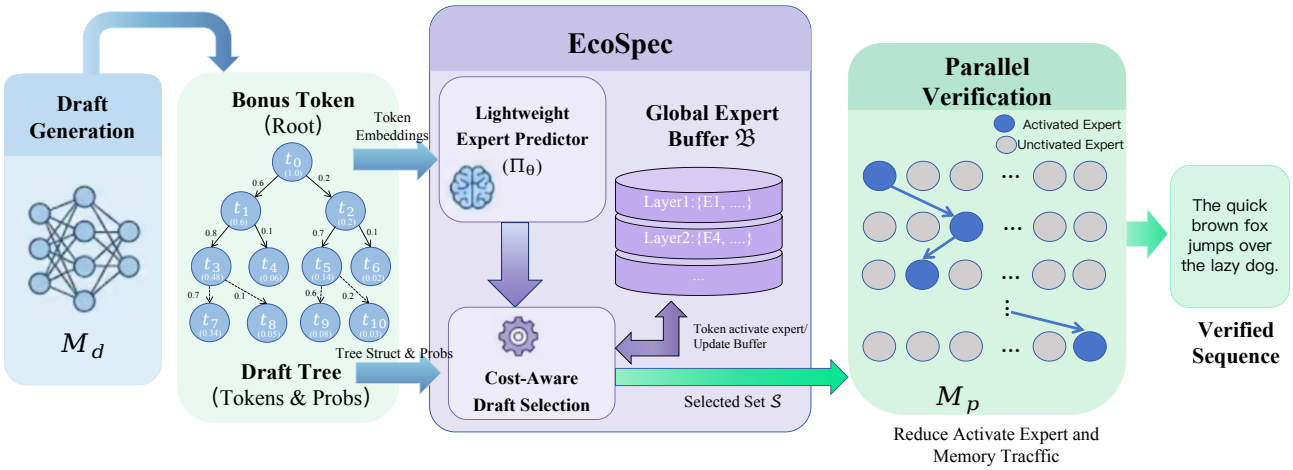


Figure 2. **Overview of the EcoSpec framework.** The process begins with draft generation, where a draft model M_d produces a tree of candidate tokens with associated probabilities. The EcoSpec module then selects a set of draft tokens $\mathcal{S}$ for verification. It employs a lightweight expert predictor Π_θ to estimate expert activations, maintains a global expert buffer $\mathcal{B}$ to track experts already covered by the selected tokens, and uses cost-aware draft selection to balance acceptance likelihood with expert activation cost. Finally, the target MoE model M_p performs parallel verification on the selected set $\mathcal{S}$.

tion latency increases with the number of unique experts activated per step, indicating that fetching additional expert weights is a major source of verification cost. Fig. 1(b) shows that standard SD selection can rapidly increase the cumulative number of unique activated experts as more tokens are selected for verification. This means that a larger verification set provides more opportunities for token acceptance, but can also make each verification pass more expensive in MoE models.

The cost of this effect is substantial at large scale. As shown in Fig. 1(c), a single expert can occupy tens of megabytes of memory, so even a small increase in the per-layer expert union can translate into large HBM traffic. For DeepSeek-V3.1, reducing the expected expert footprint by only 0.2 experts per layer per step decreases expert-weight traffic by approximately 0.5 GB per speculative step. These findings motivate EcoSpec, which incorporates expert activation cost into draft selection to reduce the verification expert footprint while maintaining high acceptance likelihood.

4. Methodology: EcoSpec

To reduce verification memory traffic in MoE speculative decoding, EcoSpec introduces a cost-aware selection mechanism between draft generation and target-model verification. Given a draft tree with candidate probabilities, EcoSpec selects γ draft tokens for parallel verification by considering both acceptance likelihood and predicted expert activation cost. As shown in Figure 2, the framework consists of three components:

(1) A lightweight **Expert Predictor** Π_θ that estimates the experts each draft token is likely to activate in the target MoE model. It outputs per-layer expert distributions, and the top-K predicted experts are used as an approximation of the token's activated expert set.

(2) A **Global Expert Buffer** $\mathcal{B}$ that records experts already covered by the selected verification set. This buffer enables efficient marginal-cost estimation by counting only the newly introduced experts when a candidate is added.

(3) A **Cost-Aware Draft Selection** algorithm that selects draft tokens from the draft tree under an acceptance–cost scoring rule. The score favors candidates with high cumulative draft probability and low marginal expert cost, encouraging selected paths to reuse experts already present in $\mathcal{B}$.

Execution order. EcoSpec runs after draft generation and before target-model verification. It first constructs a verification subset $\mathcal{S}$ of size γ using draft probabilities and predicted expert footprints, without querying the target router during this stage. During subset construction, the buffer $\mathcal{B}$ is updated from predicted path-level expert sets to estimate the marginal cost of the remaining candidates. Once $\mathcal{S}$ is constructed, the selected nodes are submitted to the standard speculative verifier and checked together in a single target-model forward pass.

4.1. Lightweight Expert Predictor

Cost-aware selection requires estimating the expert footprint of each draft token before target-model verification. Directly querying the target router for every draft candidate would

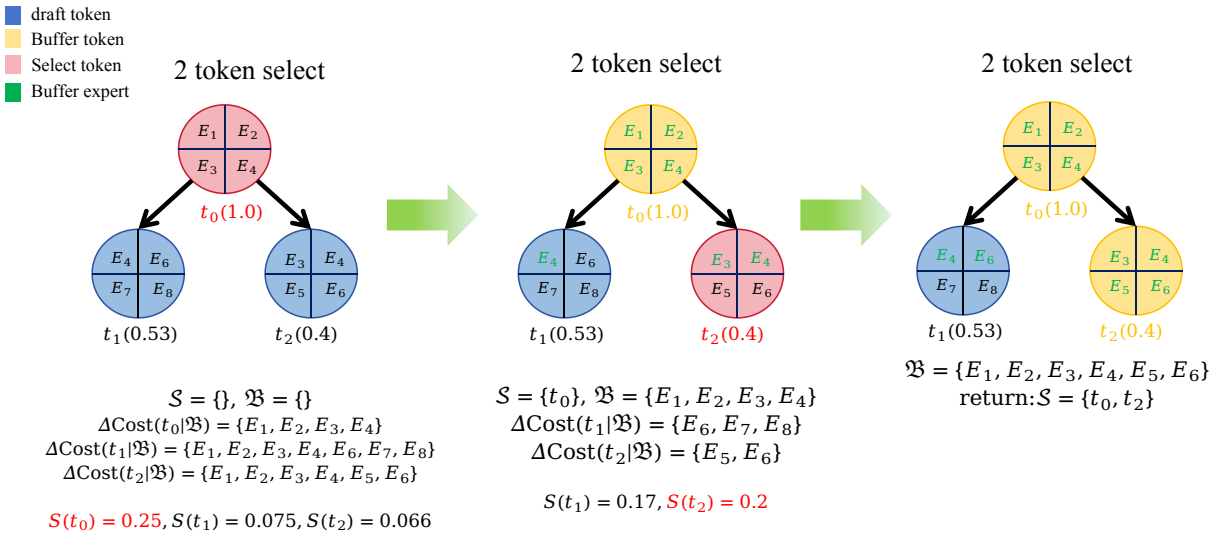


Figure 3. Illustration of Pre-verification Cost-Aware Subset Construction. We select $\gamma = 2$ draft tokens before target-model verification. Left (Step 1): the root token t_0 is selected according to the acceptance–cost score, and its predicted expert footprint $\{E_1, E_2, E_3, E_4\}$ is added to the expert buffer $\mathcal{B}$ for subsequent scoring. Middle (Step 2): the algorithm re-evaluates the remaining candidates using the updated buffer. Although t_1 has higher cumulative draft probability ($P = 0.53$), it introduces three new predicted experts ($\Delta\text{Cost} = 3$). In contrast, t_2 reuses experts already covered by $\mathcal{B}$ and introduces only two new predicted experts ($\Delta\text{Cost} = 2$), so it obtains a higher score ($S = 0.20 > 0.17$) and is selected. Right (Final): the final selected set is $\mathcal{S} = \{t_0, t_2\}$, illustrating how marginal expert cost can change the ranking induced by cumulative draft probability.

require running the target MoE model, which would offset the benefit of speculative decoding. We therefore train a lightweight expert predictor Π_θ to approximate target-model routing and provide a low-cost estimate of expert activation.

Architecture. We instantiate Π_θ with a small decoder-only backbone. For compatibility with the target MoE model M_p, each input token is first mapped by the target embedding layer and then projected into the predictor hidden space. Given a draft token t_i with context $t_{<i}$, the predictor outputs routing logits

$$\mathbf{z}_{t_i} = \Pi_\theta(t_i \mid t_{<i}) \in \mathbb{R}^{L \times E}, \tag{2}$$

where L is the number of MoE layers and E is the number of experts per layer. We obtain a routing distribution for each MoE layer by applying softmax over experts:

$$\hat{\mathbf{p}}_{t_i,\ell} = \text{Softmax}(\mathbf{z}_{t_i,\ell}) \in \mathbb{R}^E, \qquad \ell \in \{1, \dots, L\}. \tag{3}$$

The predicted expert set is then constructed as a set of layer–expert pairs:

$$\mathcal{E}_{\text{pred}}(t_i) = \{(\ell, e) \mid e \in \text{TopK}(\hat{\mathbf{p}}_{t_i,\ell}, K), \ell \in \{1, \dots, L\}\}. \tag{4}$$

Training Objective. During predictor training, we run the target MoE model offline and record the ground-truth activated experts for each token. For token t and MoE layer ℓ, let $\mathcal{E}_{\text{gt}}(t, \ell)$ denote the top-K experts selected by the target router. We define a normalized target distribution $\mathbf{q}_{t,\ell} \in [0, 1]^E$ by assigning uniform probability mass to the activated experts:

$$q_{t,\ell,e} = \begin{cases} \frac{1}{K}, & e \in \mathcal{E}_{\text{gt}}(t, \ell), \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

We train Π_θ with layer-wise cross-entropy:

$$\mathcal{L}_{\text{pred}}(\theta) = -\sum_t \sum_{\ell=1}^{L} \sum_{e=1}^{E} q_{t,\ell,e} \log \hat{p}_{t,\ell,e}. \tag{6}$$

4.2. Cost-Aware Draft Selection

Given the predicted expert sets from § 4.1, EcoSpec selects draft-tree nodes for verification under an acceptance–cost trade-off. The selection is path-based: each candidate is scored using the cumulative draft probability and predicted expert footprint of its root-to-node path. Existing draft-tree methods primarily rank candidates by confidence or acceptance likelihood (Li et al., 2024b). For MoE verification, however, a high-confidence candidate can still introduce many new experts if its routing footprint has little overlap with the candidates already selected. Therefore, EcoSpec augments confidence-based selection with the marginal expert cost induced by each candidate.

Selection State. Let $\mathcal{T}$ denote the draft tree, where each node t_i corresponds to a candidate token. During selection, EcoSpec maintains a selected set $\mathcal{S}$ and an expert buffer $\mathcal{B}$. The selected set $\mathcal{S}$ contains tokens chosen for verification, while $\mathcal{B}$ contains the experts already covered by the current selected set. The buffer is used to measure how many new experts a candidate would introduce beyond those already covered.

Path-Dependent Expert Footprint. Because draft-tree verification is prefix-dependent, selecting a node requires covering its path from the root. For a candidate node t_i, let $\mathrm{Path}(\mathrm{root} \to t_i)$ denote the sequence of nodes from the root to t_i. The predicted expert footprint of this path is

$$\mathcal{E}_{traj}(t_i) = \bigcup_{\tau \in \mathrm{Path}(\mathrm{root} \to t_i)} \mathcal{E}_{pred}(\tau). \tag{7}$$

The marginal expert cost of selecting t_i under the current buffer is then defined as

$$\Delta\mathrm{Cost}(t_i \mid \mathcal{B}) = \left| \mathcal{E}_{traj}(t_i) \setminus \mathcal{B} \right|. \tag{8}$$

This cost counts only the additional experts that are not already covered by previously selected candidates.

Acceptance–Cost Scoring. Let $P(t_i)$ be the cumulative draft probability along the path from the root to t_i. EcoSpec scores each candidate by

$$S(t_i) = \frac{P(t_i)}{\Delta\mathrm{Cost}(t_i \mid \mathcal{B}) + \epsilon}, \tag{9}$$

where ϵ avoids division by zero. This score favors candidates with high acceptance likelihood while penalizing those that introduce many new experts. Equivalently, it encourages the selected verification set to reuse experts already present in $\mathcal{B}$, thereby limiting the growth of the per-step expert union.

Prefix Consistency. The score is naturally compatible with the prefix structure of the draft tree. For a parent node t_p and its child t_c, the cumulative probability satisfies $P(t_c) = P(t_p)P(t_c \mid t_p) \le P(t_p)$, while the path footprint satisfies $\mathcal{E}_{traj}(t_p) \subseteq \mathcal{E}_{traj}(t_c)$. Therefore, $\Delta\mathrm{Cost}(t_p \mid \mathcal{B}) \le \Delta\mathrm{Cost}(t_c \mid \mathcal{B})$. Under the same buffer $\mathcal{B}$, a child node therefore cannot receive a higher score than its parent except in tie cases. Thus, the scoring rule is aligned with prefix-closed draft-tree verification without requiring an additional structural penalty.

Dynamic Buffer Effect. After a candidate path is selected, its predicted experts are added to $\mathcal{B}$. This update reduces the marginal cost of later candidates that share experts with the selected path, increasing their scores without changing their draft probabilities. As a result, EcoSpec tends to extend high-probability paths that also preserve expert locality, rather than expanding the draft tree solely by confidence. The full selection procedure is provided in Appendix Alg. 1.

5. Experiments

5.1. Setup

Models and Benchmarks. We evaluate EcoSpec on three large-scale MoE models with different sizes and routing configurations: DeepSeek-V3.1 (DeepSeek-AI et al., 2025) (671B total / 37B active, Top-8), Qwen3-235B-A22B (Yang et al., 2025) (235B total / 22B active, Top-8), and GPT-OSS-120B (OpenAI et al., 2025) (120B total / 5.1B active, Top-4). We use seven benchmarks covering mathematical reasoning, code generation, question answering, and dialogue: GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), AIME-25 (MAA, 2025), Math500 (Hendrycks et al., 2021; Lightman et al., 2024), AMC22-24 (AI-MO), MTBench (Zheng et al., 2023), and MMStar (Chen et al., 2024).

Baselines and Hardware. We compare EcoSpec with autoregressive (AR) decoding and the corresponding speculative decoding baseline for each target model: MTP for DeepSeek-V3.1, and EAGLE-3 for Qwen3-235B-A22B and GPT-OSS-120B[1]. All experiments are conducted on a server with 8 NVIDIA H200 GPUs.

Speculative Decoding Configuration. To isolate the effect of cost-aware selection, we keep the draft-generation configuration fixed between each speculative baseline and its EcoSpec variant. The draft process runs for 3 forward steps and keeps the top-2 tokens at each step to construct the draft tree. The verification budget γ is set to 4, where γ denotes the total number of tokens verified per speculative step, including the bonus token. This setting follows the operating regime of the large-scale MoE speculative baselines evaluated in this work, and the same configuration is used for the baseline and EcoSpec to ensure a controlled comparison. Appendix C further evaluates different verification budgets and shows that $\gamma = 4$ gives the highest average end-to-end speedup in our setting. All main experiments use batch size 1 unless explicitly stated otherwise. Table 1 reports both greedy target decoding (T=0) and sampling-based target decoding (T=1), following common speculative decoding evaluation practice (Li et al., 2024a). For the latency breakdown and HBM-traffic analysis in § 5.3, we use the greedy target-decoding rows of Table 1, i.e.,

[1] https://huggingface.co/lmsys/Qwen3-235B-A22B-EAGLE3 and https://huggingface.co/lmsys/EAGLE3-gpt-oss-120b-bf16.

Table 1. Main results across target models, benchmarks, and decoding temperatures. Each entry reports end-to-end speedup relative to AR decoding / mean acceptance length α / average active experts $\mathcal{E}$. Here, $\mathcal{E}$ denotes the average number of unique experts activated per MoE layer within one verification step. Rows are grouped by target MoE model and target-model decoding temperature.

Method	GSM8K Spd↑ / Len↑ / Exp↓	HumanEval Spd↑ / Len↑ / Exp↓	MMStar Spd↑ / Len↑ / Exp↓	AIME-25 Spd↑ / Len↑ / Exp↓	Math500 Spd↑ / Len↑ / Exp↓	AMC22-24 Spd↑ / Len↑ / Exp↓	MTBench Spd↑ / Len↑ / Exp↓	Average Spd↑ / Len↑ / Exp↓
Qwen3-235B-A22B (Total 235B, Active 22B, Top-8)								
Temperature = 0 (Greedy)								
AR	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0
Eagle-3	1.31 / 2.54 / 22.3	1.11 / 2.35 / 26.2	1.35 / 2.64 / 20.5	1.14 / 2.41 / 22.1	1.05 / 1.96 / 25.4	1.12 / 2.20 / 25.4	1.46 / 2.78 / 24.0	1.22 / 2.41 / 23.7
EcoSpec	**1.39** / 2.54 / **21.0**	**1.33** / 2.32 / **21.6**	**1.57** / 2.58 / **18.4**	**1.23** / 2.37 / **20.9**	**1.13** / 1.88 / **19.4**	**1.28** / 2.10 / **22.2**	**1.62** / 2.44 / **19.9**	**1.36** / 2.32 / **20.5**
Temperature = 1 (Sampling)								
AR	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0
Eagle-3	1.28 / 2.59 / 22.8	1.17 / 2.21 / 24.3	1.49 / 2.88 / 22.4	1.13 / 2.26 / 21.8	1.17 / 2.19 / 24.9	1.28 / 2.25 / 25.0	1.45 / 2.58 / 26.5	1.28 / 2.42 / 24.0
EcoSpec	**1.37** / 2.45 / **20.3**	**1.27** / 2.20 / **21.4**	**1.62** / 2.74 / **19.9**	**1.22** / 2.23 / **20.7**	**1.25** / 2.13 / **20.8**	**1.32** / 2.20 / **22.20**	**1.61** / 2.56 / **21.3**	**1.38** / 2.36 / **20.9**
GPT-OSS-120B (Total 120B, Active 5.1B, Top-4)								
Temperature = 0 (Greedy)								
AR	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0
Eagle-3	1.05 / 1.56 / 11.9	1.05 / 1.72 / 11.8	1.28 / 1.91 / 11.7	1.17 / 2.02 / 12.2	1.14 / 2.02 / 12.0	1.14 / 2.02 / 11.9	1.18 / 1.91 / 10.3	1.14 / 1.88 / 11.6
EcoSpec	**1.11** / 1.52 / **10.6**	**1.26** / 1.72 / **10.7**	**1.45** / 1.90 / **10.5**	**1.33** / 2.02 / **10.8**	**1.27** / 2.00 / **10.6**	**1.45** / 2.02 / **10.9**	**1.30** / 1.81 / **9.6**	**1.31** / 1.86 / **10.6**
Temperature = 1 (Sampling)								
AR	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0	1.00× / - / 4.0
Eagle-3	1.10 / 1.90 / 11.8	1.27 / 2.13 / 11.8	1.31 / 2.20 / 11.3	1.19 / 2.13 / 12.1	1.17 / 2.08 / 12.1	1.17 / 2.22 / 12.1	1.07 / 2.17 / 12.4	1.18 / 2.12 / 11.9
EcoSpec	**1.15** / 1.85 / **10.3**	**1.34** / 2.06 / **10.8**	**1.50** / 2.15 / **10.5**	**1.27** / 2.11 / **11.5**	**1.27** / 2.06 / **10.8**	**1.36** / 2.08 / **11.4**	**1.23** / 2.06 / **11.1**	**1.30** / 2.05 / **10.9**
DeepSeek-V3.1 (Total 671B, Active 37B, Top-8)								
Temperature = 0 (Greedy)								
AR	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0
MTP	1.13 / 3.19 / 31.9	1.06 / 2.82 / 31.8	1.09 / 2.61 / 31.5	1.05 / 3.13 / 31.3	1.12 / 2.73 / 31.1	1.20 / 3.05 / 31.5	1.04 / 2.41 / 31.1	1.10 / 2.85 / 31.4
EcoSpec	**1.19** / 3.13 / **31.7**	**1.10** / 2.80 / **31.6**	**1.13** / 2.55 / **31.2**	**1.08** / 3.03 / **31.2**	**1.19** / 2.64 / **30.9**	**1.28** / 3.00 / **31.3**	**1.09** / 2.38 / **30.9**	**1.15** / 2.79 / **31.2**
Temperature = 1 (Sampling)								
AR	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0	1.00× / - / 8.0
MTP	1.22 / 3.72 / 31.8	1.29 / 3.39 / 31.9	1.28 / 3.37 / 31.7	1.26 / 3.49 / 31.6	1.24 / 3.61 / 31.1	1.44 / 3.68 / 31.1	1.20 / 3.15 / 30.8	1.28 / 3.49 / 31.4
EcoSpec	**1.35** / 3.53 / **31.6**	**1.31** / 3.32/ **30.9**	**1.35** / 3.36 / **31.4**	**1.30** / 3.43 / **31.3**	**1.29** / 3.61 / **30.8**	**1.47** / 3.59 / **30.1**	**1.23** / 3.04 / **30.5**	**1.33** / 3.41 / **30.9**

batch size 1, $\gamma = 4$, and $T{=}0$. We further evaluate larger batch sizes, different verification budgets γ, and different predictor accuracy levels in Appendix A, Appendix C, and Appendix B.2, respectively.

Expert Predictors. EcoSpec reuses the draft-generation infrastructure of the corresponding speculative baseline and only modifies the draft selection stage. For expert-cost estimation, we use DeepSeek-R1-Distill-Qwen-1.5B (Guo et al., 2025) as the predictor backbone for DeepSeek-V3.1, and Qwen3-0.6B (Yang et al., 2025) for Qwen3-235B-A22B and GPT-OSS-120B. Predictor training details are provided in Appendix B.1.

Inference Backend. The main experiments are conducted with a HuggingFace Transformers (Wolf et al., 2020) research prototype following the EAGLE-3 inference pipeline. We use this prototype to keep the draft-generation, tree-verification, and KV-cache update workflow consistent with the released EAGLE-3 implementation, so that the comparison isolates the effect of EcoSpec's cost-aware draft selection. During decoding, the draft model constructs the draft tree, and the selected draft nodes are verified by the target MoE model in a single forward pass with a tree-structured attention mask. EcoSpec only adds the expert predictor and cost-aware selection before this verification step; the target-model verification rule is unchanged.

Table 2. Estimated HBM read traffic during verification. We report the estimated total HBM read bytes during the verification phase of one speculative step.

Model	Baseline	Baseline HBM (GB)	EcoSpec HBM (GB)	Reduction (GB / %)
Qwen3-235B-A22B	EAGLE-3	99.3	88.1	11.2 / 11.3%
GPT-OSS-120B	EAGLE-3	6.0	5.5	0.5 / 8.0%
DeepSeek-V3.1	MTP	97.3	96.8	0.5 / 1.0%

5.2. Main Results

Table 1 reports end-to-end speedup relative to autoregressive decoding, mean acceptance length α, and the average number of unique experts $\mathcal{E}$ activated per MoE layer within one verification step. Across three MoE backbones and seven benchmarks, EcoSpec improves decoding speed over the corresponding speculative baseline while consistently reducing the expert footprint. The acceptance lengths remain close to the baselines, indicating that the speedup mainly comes from reducing verification cost rather than increasing the number of accepted tokens.

Greedy decoding ($T{=}0$). Under greedy decoding, EcoSpec consistently improves speedup across all three MoE models. For Qwen3-235B-A22B, EcoSpec improves the average speedup from EAGLE-3's $1.22\times$ to $1.36\times$, while reducing $\mathcal{E}$ from 23.7 to 20.5. The largest gain appears on MTBench, where EcoSpec reaches $1.62\times$ speedup. For GPT-OSS-120B, EcoSpec increases the average speedup from $1.14\times$ to $1.31\times$ and reduces $\mathcal{E}$ from 11.6 to 10.6. On

Table 3. Latency breakdown and throughput analysis. We report the average wall-clock time in seconds per speculative step. $\mathcal{E}$ denotes the average number of unique experts activated per MoE layer within one verification step. T_{pred}, T_{draft}, T_{verify}, and T_{total} denote predictor overhead, draft-generation time, target verification time, and total speculative-step latency, respectively. α denotes the mean accepted tokens per step. The speedup column reports the corresponding end-to-end throughput speedup under the same $T = 0$ setting as Table 1. The latency-breakdown columns are reported to explain the sources of the end-to-end speedup.

Model	Method	Experts ($\mathcal{E}$) ↓	Predictor (T_{pred})	Drafting (T_{draft})	Verify (T_{verify}) ↓	Total Step (T_{total}) ↓	Accept Len (α) ↑	Throughput Speedup ↑
Qwen3-235B-A22B	Baseline (AR)	8.0	–	–	–	0.490s†	1.00	1.00×
	EAGLE-3	23.7	–	0.008s	0.832s	0.840s	2.41	1.22×
	EcoSpec	20.5	0.004s	0.008s	0.730s	0.742s	2.32	1.36×
	Δ vs. EAGLE-3	-3.2	+0.004s	0.000s	-0.102s	-0.098s	-0.09	+0.14×
GPT-OSS-120B	Baseline (AR)	4.0	–	–	–	0.081s†	1.00	1.00×
	EAGLE-3	11.6	–	0.035s	0.113s	0.148s	1.88	1.14×
	EcoSpec	10.6	0.004s	0.035s	0.090s	0.129s	1.86	1.31×
	Δ vs. EAGLE-3	-1.0	+0.004s	0.000s	-0.023s	-0.019s	-0.02	+0.17×
DeepSeek-V3.1	Baseline (AR)	8.0	–	–	–	0.369s†	1.00	1.00×
	MTP	31.4	–	0.008s	0.980s	0.988s	2.85	1.10×
	EcoSpec	31.2	0.004s	0.008s	0.930s	0.942s	2.79	1.15×
	Δ vs. MTP	-0.2	+0.004s	0.000s	-0.050s	-0.046s	-0.06	+0.05×

† AR baseline time is per-token latency. Speculative methods report per-step latency and generate α accepted tokens per step. Delta rows compare EcoSpec with the corresponding speculative baseline.

MMStar and AMC22-24, the speedup reaches $1.45\times$. For DeepSeek-V3.1, EcoSpec improves the average speedup from MTP's $1.10\times$ to $1.15\times$, while reducing $\mathcal{E}$ from 31.4 to 31.2.

Sampling decoding (T=1). Under sampling, draft candidates become more diverse, but EcoSpec continues to reduce expert activation and improve speedup. For Qwen3-235B-A22B, the average speedup increases from EAGLE-3's $1.28\times$ to $1.38\times$, while $\mathcal{E}$ decreases from 24.0 to 20.9. For GPT-OSS-120B, the average speedup improves from $1.18\times$ to $1.30\times$, with $\mathcal{E}$ reduced from 11.9 to 10.9. For DeepSeek-V3.1, EcoSpec improves the average speedup from MTP's $1.28\times$ to $1.33\times$ and reduces $\mathcal{E}$ from 31.4 to 30.9. These results show that the acceptance–cost trade-off remains effective under both greedy and sampling-based target decoding regimes.

Backbone-dependent gains. The magnitude of the gain varies across MoE backbones. Qwen3-235B-A22B and GPT-OSS-120B show larger reductions in active experts, suggesting more opportunity for expert reuse during draft selection. DeepSeek-V3.1 shows smaller expert-footprint reductions, which is consistent with its more balanced routing pattern analyzed in Appendix D. Nevertheless, because DeepSeek-V3.1 has a large per-expert memory footprint, even a reduction of 0.2 experts per layer can translate into approximately 0.5 GB less expert-weight traffic per speculative step. Thus, EcoSpec remains beneficial even when the available expert-reuse headroom is smaller.

5.3. Latency Breakdown and Overhead Analysis

To understand why EcoSpec improves performance, we analyze the HBM traffic and latency composition of a speculative decoding step in Table 2 and Table 3. Following the evaluation scope defined in § 5.1, this subsection analyzes the latency and HBM-traffic behavior of the T=0 runs. We decompose the total speculative-step latency into three parts: (1) draft generation (T_{draft}), (2) expert prediction overhead introduced by EcoSpec (T_{pred}), and (3) target-model verification of the selected draft tokens (T_{verify}). EcoSpec does not reduce the computation required by each activated expert. Instead, its speedup comes from reducing expert-weight memory traffic during verification.

HBM Read Traffic. We estimate HBM read traffic during verification from the activated expert footprint and the model-specific expert size. As shown in Table 2, EcoSpec reduces the estimated HBM reads from 99.3 GB to 88.1 GB on Qwen3-235B-A22B, saving 11.2 GB per speculative step. For GPT-OSS-120B, the estimated reads decrease from 6.0 GB to 5.5 GB. For DeepSeek-V3.1, the reduction is smaller, from 97.3 GB to 96.8 GB, which is consistent with its smaller expert-reuse headroom. These reductions correspond to the lower verification latency reported in Table 3.

Verification Latency and Predictor Overhead. Table 3 shows that EcoSpec reduces verification latency across all three MoE models while adding only a small predic-

Table 4. Ablation study on GSM8K. We compare the static global-cost variant with EcoSpec's marginal-cost scoring.

Model	Strategy	Speedup ↑	Len (α) ↑	Avg. $\mathcal{E}$ ↓
Qwen3-235B-A22B	Global Cost	1.28×	2.21	21.5
	EcoSpec	**1.39**×	**2.54**	**21.0**
GPT-OSS-120B	Global Cost	1.06×	1.35	10.8
	EcoSpec	**1.11**×	**1.52**	**10.6**
DeepSeek-V3.1	Global Cost	1.16×	2.95	31.8
	EcoSpec	**1.19**×	**3.13**	**31.7**

tor overhead. On Qwen3-235B-A22B, the average active expert count decreases from 23.7 to 20.5, and T_{verify} decreases from 0.832s to 0.730s. On GPT-OSS-120B, the active expert count decreases from 11.6 to 10.6, and T_{verify} decreases from 0.113s to 0.090s. On DeepSeek-V3.1, the expert-footprint reduction is smaller (31.4 → 31.2), but T_{verify} still decreases from 0.980s to 0.930s. Across all three models, the predictor overhead is about 4ms per speculative step, which is small relative to the reduction in verification latency. The acceptance length changes only slightly, so the speedup is mainly explained by reduced verification cost rather than higher acceptance length.

5.4. Ablation Study: Impact of Marginal Cost Scoring

We study the effect of marginal-cost scoring on GSM8K across all three target models. We compare EcoSpec with a static *Global Cost* variant, where each node is scored by the accumulated predicted expert footprint along its path:

$$S_{\text{global}}(t_i) = \frac{P(t_i)}{|\mathcal{E}_{traj}(t_i)| + \epsilon}. \tag{10}$$

Unlike EcoSpec, this variant does not update the expert buffer during selection and therefore cannot discount experts that are already covered by previously selected nodes. As a result, deeper nodes tend to receive larger accumulated costs even when they reuse experts from earlier selected paths.

Table 4 shows that marginal-cost scoring consistently improves speedup over the static global-cost variant. On Qwen3-235B-A22B, EcoSpec increases speedup from 1.28× to 1.39× and mean acceptance length from 2.21 to 2.54, while reducing the average active experts from 21.5 to 21.0. A similar pattern appears on GPT-OSS-120B, where speedup improves from 1.06× to 1.11× and acceptance length increases from 1.35 to 1.52. On DeepSeek-V3.1, the gain is smaller but still consistent. These results indicate that updating the buffer during selection helps EcoSpec identify candidates that extend accepted paths while reusing already covered experts.

5.5. Additional Baseline Evaluation

The main experiments compare EcoSpec with the corresponding speculative decoding baselines for each target model: MTP for DeepSeek-V3.1, and EAGLE-3 for Qwen3-235B-A22B and GPT-OSS-120B. To provide an additional baseline comparison, we further include Group Tree Optimization (GTO) (Hu et al., 2026), which improves EAGLE-style draft models by better aligning draft training with tree-based decoding. GTO and EcoSpec act on different stages of the speculative decoding pipeline: GTO improves draft generation, while EcoSpec changes the pre-verification selection of draft nodes.

Table 5. Additional baseline evaluation with GTO. We report end-to-end speedup and acceptance length for GTO and GTO+EcoSpec.

Model	Dataset	GTO Spd.	GTO Len	GTO+EcoSpec Spd.	GTO+EcoSpec Len
Qwen3-235B-A22B	GSM8K	1.28×	2.50	1.34×	2.50
Qwen3-235B-A22B	HumanEval	1.13×	2.41	1.29×	2.37
GPT-OSS-120B	GSM8K	1.03×	1.53	1.08×	1.50
GPT-OSS-120B	HumanEval	1.08×	1.77	1.27×	1.76

For this experiment, we start from the released EAGLE-3 draft model and continue training it with the GTO procedure. The resulting GTO-trained drafter is used to generate candidate trees under the standard speculative verification workflow. We then evaluate GTO+EcoSpec by applying the same cost-aware draft-selection strategy at the verification stage.

As shown in Table 5, GTO+EcoSpec achieves higher end-to-end speedup than GTO on all evaluated settings. This additional baseline comparison further supports the effectiveness of EcoSpec.

6. Conclusion

We presented EcoSpec, a cost-aware speculative decoding framework for large-scale MoE models. EcoSpec addresses expert scattering during speculative verification by incorporating predicted marginal expert activation cost into draft-tree selection. With a lightweight expert predictor and a dynamic expert buffer, EcoSpec selects draft tokens that preserve acceptance likelihood while reducing the growth of the verification expert footprint, without modifying the target-model verification rule. Experiments on DeepSeek-V3.1, Qwen3-235B-A22B, and GPT-OSS-120B show consistent speedups and reduced active experts across reasoning, coding, and dialogue benchmarks, with up to 1.62× speedup. These results highlight the importance of accounting for MoE-specific expert activation costs when applying speculative decoding to large-scale sparse models.

References

AI-MO. Aimo validation amc dataset. URL https://huggingface.co/datasets/AI-MO/aimo-validation-amc. Hugging Face.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=PEpbUobfJv.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Chen, L., Li, J., Dong, X., Zhang, P., Zang, Y., Chen, Z., Duan, H., Wang, J., Qiao, Y., Lin, D., and Zhao, F. Are we on the right way for evaluating large vision-language models? In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=evP9mxNNxJ.

Chen, L., Wen, Z., Wu, T., Zhang, X., and Wu, C. Sp-moe: Speculative decoding and prefetching for accelerating moe-based model inference, 2025. URL https://arxiv.org/abs/2510.10302.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=H4DqfPSibmx.

DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., et al. Deepseek-v3 technical report, 2025. URL https://arxiv.org/abs/2412.19437.

Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., Zoph, B., Fedus, L., Bosma, M. P., Zhou, Z., Wang, T., Wang, E., Webster, K., Pellat, M., Robinson, K., Meier-Hellstern, K., Duke, T., Dixon, L., Zhang, K., Le, Q., Wu, Y., Chen, Z., and Cui, C. GLaM: Efficient scaling of language models with mixture-of-experts. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 5547–5569. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/du22c.html.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), January 2022. ISSN 1532-4435.

Gale, T., Narayanan, D., Young, C., and Zaharia, M. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. *Proceedings of Machine Learning and Systems*, 5, 2023.

Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Hu, S., Li, J., Lu, Z., and Zhou, P. Bridging draft policy misalignment: Group tree optimization for speculative decoding. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=dwPdYFqVWO.

Huang, H., Ardalani, N., Sun, A., Ke, L., Bhosale, S., Lee, H.-H. S., Wu, C.-J., and Lee, B. Toward efficient inference for mixture of experts. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=stXtBqyTWX.

Huang, Z., Zhu, L., Zhan, Z., Hu, T., Mao, W., Yu, X., Liu, Y., and Zhang, T. MoESD: Unveil speculative decoding's potential for accelerating sparse moe. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=FAeU7516MR.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. {GS}hard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Forty-first International Conference on Machine Learning*, 2024a. URL https://openreview.net/forum?id=1NdN7eXyb4.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7421–7432, Miami, Florida, USA, November 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.422. URL https://aclanthology.org/2024.emnlp-main.422/.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=4exx1hUffq.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=v8L0pN6EOi.

Liu, J., Tang, P., Wang, W., Ren, Y., Hou, X., Heng, P. A., Guo, M., and Li, C. A survey on inference optimization techniques for mixture of experts models. *ACM Comput. Surv.*, 58(10), March 2026. ISSN 0360-0300. doi: 10.1145/3794845. URL https://doi.org/10.1145/3794845.

MAA. American invitational mathematics examination - AIME, 2025. URL https://huggingface.co/datasets/opencompass/AIME2025. Hugging Face.

McDanel, B., Li, S., Surineni, S., and Khaitan, H. Moe-spec: Expert budgeting for efficient speculative decoding, 2026. URL https://arxiv.org/abs/2602.16052.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, pp. 932–949, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400703867. doi: 10.1145/3620666.3651335. URL https://doi.org/10.1145/3620666.3651335.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., et al. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

OpenAI, :, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., et al. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '20. IEEE Press, 2020. ISBN 9781728199986.

Shazeer, N. Fast transformer decoding: One write-head is all you need, 2019. URL https://arxiv.org/abs/1911.02150.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=B1ckMDqlg.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Wang, W., Liu, J., Hou, X., Xia, X., Tang, P., Zhang, M., Li, C., and Guo, M. Moe-speq: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts, 2025. URL https://arxiv.org/abs/2511.14102.

Williams, S., Waterman, A., and Patterson, D. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, April 2009. ISSN 0001-0782. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL https://aclanthology.org/2020.emnlp-demos.6/.

Xue, F., Zheng, Z., Fu, Y., Ni, J., Zheng, Z., Zhou, W., and You, Y. Openmoe: an early effort on open mixture-of-experts language models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Xue, L., Fu, Y., Lu, Z., Mai, L., and Marina, M. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache, 2025. URL https://arxiv.org/abs/2401.14361.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL https://openreview.net/forum?id=uccHPGDlao.

Algorithm 1 Cost-Aware Draft Selection in EcoSpec

Input: Draft Tree $\mathcal{T}$, Budget γ, Predictor Π_θ, Initial Buffer $\mathcal{B}_0$
Output: Selected tokens $\mathcal{S}$
$\mathcal{S} \leftarrow \emptyset$
$\mathcal{B} \leftarrow \mathcal{B}_0$
Construct the verification subset before target-model verification.
while $|\mathcal{S}| < \gamma$ **do**
  for each unselected node $t_i \in \mathcal{T}$ **do**
    $\mathcal{E}_{traj}(t_i) \leftarrow \bigcup_{\tau \in \text{Path}(root \to t_i)} \mathcal{E}_{pred}(\tau)$
    $Cost_i \leftarrow |\mathcal{E}_{traj}(t_i) \setminus \mathcal{B}|$
    $Score_i \leftarrow P(t_i)/(Cost_i + \epsilon)$
  end for
  $t^* \leftarrow \arg\max_{t_i} Score_i$
  $\mathcal{S} \leftarrow \mathcal{S} \cup \{t^*\}$
  $\mathcal{B} \leftarrow \mathcal{B} \cup \mathcal{E}_{traj}(t^*)$ {Update buffer for subsequent scoring}
end while
return $\mathcal{S}$ {Verify all selected nodes in one target forward pass}

A. Impact of Batch Size on Scalability

A.1. End-to-End Batch-Size Scaling

Batching is a common way to improve GPU utilization during inference. To examine how batching affects EcoSpec, we compare it with EAGLE-3 under batch sizes $B = \{1, 2, 4\}$ and report end-to-end speedup together with active expert counts.

As shown in Tables 6 and 7, both methods exhibit lower speedup as batch size increases. On Qwen3-235B-A22B, EAGLE-3's average speedup drops from $1.22\times$ at $B = 1$ to $1.00\times$ at $B = 4$, while EcoSpec maintains a higher average speedup of $1.09\times$ at $B = 4$. A similar trend appears on GPT-OSS-120B, where EAGLE-3 drops from $1.14\times$ to $1.00\times$, while EcoSpec retains $1.06\times$ at $B = 4$.

Active expert counts help explain this speedup gap. As batch size increases, each verification step covers more candidate tokens in parallel, expanding the union of activated experts and increasing verification cost. At $B = 4$ on Qwen3-235B-A22B, EAGLE-3 activates an average of 54.9 experts per step, while EcoSpec reduces this number to 45.7. On GPT-OSS-120B, the corresponding counts are 32.6 for EAGLE-3 and 28.1 for EcoSpec. This smaller expert footprint lowers verification cost relative to EAGLE-3, allowing EcoSpec to retain higher end-to-end speedup under batching.

Table 6. Batch-size scaling on Qwen3-235B-A22B. We report end-to-end speedup relative to AR decoding and the average number of active experts.

Dataset	Method	BS = 1		BS = 2		BS = 4	
		Spd ↑	Exp ↓	Spd ↑	Exp ↓	Spd ↑	Exp ↓
GSM8K	EAGLE-3	1.31	22.3	1.18	42.9	1.00	53.5
	EcoSpec	**1.39**	**21.0**	**1.24**	**35.5**	**1.07**	**45.3**
HumanEval	EAGLE-3	1.11	26.2	1.05	46.8	0.95	57.5
	EcoSpec	**1.33**	**21.6**	**1.21**	**36.0**	**1.08**	**47.8**
MMStar	EAGLE-3	1.35	20.5	1.22	41.0	1.05	51.8
	EcoSpec	**1.57**	**18.4**	**1.40**	**34.1**	**1.18**	**39.9**
AIME-25	EAGLE-3	1.14	22.1	1.08	42.6	0.98	53.0
	EcoSpec	**1.23**	**20.9**	**1.15**	**31.3**	**1.06**	**41.9**
Math500	EAGLE-3	1.05	25.4	1.02	46.0	0.98	56.5
	EcoSpec	**1.13**	**19.4**	**1.09**	**39.8**	**1.03**	**47.5**
AMC22-24	EAGLE-3	1.12	25.4	1.07	47.8	1.02	56.5
	EcoSpec	**1.28**	**22.2**	**1.14**	**38.6**	**1.04**	**49.3**
MTBench	EAGLE-3	1.46	24.0	1.25	44.8	1.05	55.5
	EcoSpec	**1.62**	**19.9**	**1.42**	**37.6**	**1.19**	**48.5**
Average	EAGLE-3	1.22	23.7	1.12	44.6	1.00	54.9
	EcoSpec	**1.36**	**20.5**	**1.24**	**36.1**	**1.09**	**45.7**

Table 7. Batch-size scaling on GPT-OSS-120B. We report end-to-end speedup relative to AR decoding and the average number of active experts.

Dataset	Method	BS = 1		BS = 2		BS = 4	
		Spd ↑	Exp ↓	Spd ↑	Exp ↓	Spd ↑	Exp ↓
GSM8K	EAGLE-3	1.05	11.9	1.02	25.1	0.93	35.4
	EcoSpec	**1.11**	**10.6**	**1.08**	**22.8**	**0.97**	**30.1**
HumanEval	EAGLE-3	1.05	11.8	1.03	22.0	0.98	32.3
	EcoSpec	**1.26**	**10.7**	**1.18**	**18.9**	**1.03**	**28.2**
MMStar	EAGLE-3	1.28	11.7	1.15	21.9	1.04	32.2
	EcoSpec	**1.45**	**10.5**	**1.30**	**18.7**	**1.15**	**26.0**
AIME-25	EAGLE-3	1.17	12.2	1.10	22.5	1.02	31.8
	EcoSpec	**1.33**	**10.8**	**1.21**	**20.1**	**1.08**	**27.4**
Math500	EAGLE-3	1.14	12.0	1.08	24.2	0.97	34.5
	EcoSpec	**1.27**	**10.6**	**1.15**	**20.8**	**1.05**	**29.0**
AMC22-24	EAGLE-3	1.14	11.9	1.07	20.7	1.03	31.3
	EcoSpec	**1.45**	**10.9**	**1.30**	**18.6**	**1.09**	**27.9**
MTBench	EAGLE-3	1.18	10.3	1.10	19.9	1.03	30.4
	EcoSpec	**1.30**	**9.6**	**1.20**	**16.6**	**1.08**	**27.9**
Average	EAGLE-3	1.14	11.6	1.08	22.3	1.00	32.6
	EcoSpec	**1.31**	**10.6**	**1.20**	**19.5**	**1.06**	**28.1**

A.2. Batch-Size Scaling of Verification Cost

We further examine larger-batch behavior using a verification-cost-oriented metric on 100 randomly sampled instances from the seven evaluation datasets. In Table 8, we report

$$\rho = \frac{T_{\mathrm{AR}}}{T_{\mathrm{verify}}},$$

where T_{AR} denotes the average AR decoding time and T_{verify} denotes the average speculative verification time per step. A larger ρ indicates that speculative verification is cheaper relative to AR decoding. We also report BS=8 Spd, the throughput of each speculative method normalized by AR throughput at batch size 8.

The ratio ρ decreases for both methods as batch size increases. On Qwen3-235B-A22B, ρ drops from $0.59\times$ to $0.32\times$ for EAGLE-3 and from $0.67\times$ to $0.33\times$ for EcoSpec. On GPT-OSS-120B, it drops from $0.72\times$ to $0.43\times$ for EAGLE-3 and from $0.90\times$ to $0.44\times$ for EcoSpec. At BS=8, both speculative methods fall below AR throughput in this prototype, with BS=8 speedup of $0.69\times$ on Qwen3-235B-A22B and $0.65\times$ on GPT-OSS-120B.

These results indicate that large-batch speculative decoding remains challenging in this prototype. Nevertheless, EcoSpec consistently maintains a higher $T_{\mathrm{AR}}/T_{\mathrm{verify}}$ ratio than EAGLE-3 across the tested batch sizes, indicating lower verification cost under the same setting.

Table 8. Batch-size scaling of verification cost. We report $\rho = T_{\mathrm{AR}}/T_{\mathrm{verify}}$, the ratio between average AR decoding time and average speculative verification time per step. BS=8 Spd reports throughput normalized by AR throughput at batch size 8.

Model	Method	BS=1	BS=2	BS=4	BS=8	BS=8 Spd
Qwen3-235B-A22B	EAGLE-3	0.59×	0.55×	0.46×	0.32×	0.69×
	EcoSpec	**0.67**×	**0.62**×	**0.52**×	**0.33**×	0.69×
GPT-OSS-120B	EAGLE-3	0.72×	0.69×	0.63×	0.43×	0.65×
	EcoSpec	**0.90**×	**0.82**×	**0.72**×	**0.44**×	0.65×

B. Predictor Details and Analysis

B.1. Predictor Training Details

EcoSpec trains a lightweight expert predictor for each target MoE model to estimate expert activation cost before target-model verification. The predictor is used only for cost-aware draft selection. It does not participate in token verification, does not query the target MoE model online, and does not modify the standard speculative verification rule.

Predictor backbones. We instantiate Π_θ with small open-source LMs and fine-tune them to predict the target model's per-layer expert activations from token-level inputs. For DeepSeek-V3.1, we use DeepSeek-R1-Distill-Qwen-1.5B as the predictor backbone. For Qwen3-235B-A22B, we use Qwen3-0.6B, leveraging architectural proximity within the Qwen family. For GPT-OSS-120B, since no lightweight model from the same series is publicly available, we also use Qwen3-0.6B. The predictor cost is small relative to target-model verification, as reflected in Table 3.

Training data and setup. We collect routing traces offline by running each target MoE model on seven datasets span-

ning reasoning, coding, and dialogue: HumanEval, MMStar, MT-Bench, AMC22-24, GSM8K, AIME-25, and Math500. For each token and MoE layer, we record the target router's Top-K selected experts as supervision, where K follows the target model's routing configuration. We randomly split samples into 80% for training and 20% for testing. All evaluations reported in the main paper are conducted on the held-out test split. We fine-tune predictors for 100 epochs using Adam with learning rate 1×10^{-5} and batch size 16.

Routing prediction accuracy. We report Top-K routing prediction accuracy on the test split, computed as the average overlap ratio between the predicted Top-K experts and the target router's Top-K experts across tokens and MoE layers. The resulting accuracies are 80% for DeepSeek-V3.1, 82% for Qwen3-235B-A22B, and 93% for GPT-OSS-120B. The higher accuracy on GPT-OSS-120B is consistent with its Top-4-over-128 routing setting, which yields more concentrated activation patterns than the Top-8 routing used by DeepSeek-V3.1 and Qwen3-235B-A22B. Since EcoSpec uses the predictor only as a cost-estimation module, exact routing reconstruction is not required; the sensitivity to predictor quality is further evaluated in Appendix B.2.

B.2. Impact of Predictor Accuracy

To examine the effect of predictor quality, we evaluate EcoSpec with three predictor checkpoints: two intermediate checkpoints with Top-K routing accuracy of approximately 50% and 60%, and the converged checkpoint used in the main experiments. The decoding configuration is fixed across all runs, including the default verification budget $\gamma = 4$; only the predictor checkpoint is changed.

Table 9 reports end-to-end speedup, acceptance length, and active expert count on Qwen3-235B-A22B and GPT-OSS-120B. Lower-accuracy predictors lead to lower speedup and higher active expert counts. On Qwen3-235B-A22B, the converged predictor improves the speedup from $1.17\times$ to $1.39\times$ on GSM8K and from $1.20\times$ to $1.62\times$ on MT-Bench compared with the 50%-accuracy checkpoint. On GPT-OSS-120B, the corresponding speedup improves from $1.01\times$ to $1.11\times$ on GSM8K and from $1.07\times$ to $1.30\times$ on MT-Bench. The active expert count also decreases consistently as predictor quality improves.

These results show that predictor accuracy affects the quality of cost-aware draft selection. The converged predictor gives the best speedup in all reported settings, while intermediate predictors still retain positive speedup under the same decoding configuration.

B.3. Oracle Expert-Set Analysis

We further analyze whether the learned predictor provides a sufficiently accurate expert-cost signal for cost-aware draft selection. In this analysis, we keep the EcoSpec selection rule unchanged and replace the predicted expert sets with ground-truth expert sets collected offline from the target MoE model. This isolates the effect of predictor error while keeping the selection rule fixed.

Table 9. Robustness to Predictor Accuracy. Predictor-accuracy sensitivity. We vary the predictor checkpoint while keeping the decoding configuration fixed.

Model	Dataset	Acc ≈ 50% Spd / Len / Exp	Acc ≈ 60% Spd / Len / Exp	Converged **Spd** / Len / Exp
Qwen3-235B	GSM8K	1.17 / 2.20 / 24.5	1.29 / 2.50 / 22.5	**1.39** / 2.54 / 21.0
	MT-Bench	1.20 / 2.45 / 25.2	1.45 / 2.50 / 23.8	**1.62** / 2.44 / 19.9
GPT-OSS-120B	GSM8K	1.01 / 1.25 / 13.5	1.04 / 1.45 / 11.6	**1.11** / 1.52 / 10.6
	MT-Bench	1.07 / 1.80 / 17.8	1.14 / 1.45 / 13.5	**1.30** / 1.81 / 9.6

Table 10. Oracle expert-set analysis. Oracle uses ground-truth target-router activations collected offline, while Predictor uses the learned expert predictor. All entries report verification-oriented metrics under the same EcoSpec selection rule.

Model	Selector	Time (s)	Accept Len	Active Experts
Qwen3-235B-A22B	Oracle	0.730	2.32	20.4
	Predictor	0.730	2.32	20.5
GPT-OSS-120B	Oracle	0.090	1.86	10.5
	Predictor	0.090	1.86	10.6
DeepSeek-V3.1	Oracle	0.929	2.79	31.2
	Predictor	0.930	2.79	31.2

The oracle expert sets are used only for analysis. In normal speculative decoding, the true expert sets of candidate draft nodes are unavailable before target-model verification unless additional target-model computation is performed. Therefore, this setting is reported with verification-oriented metrics rather than end-to-end throughput, and is not used as an online inference baseline.

As shown in Table 10, replacing the learned predictor with oracle expert sets produces nearly unchanged acceptance lengths, verification times, and active expert counts. This indicates that the converged predictor provides a cost signal close to that obtained from oracle expert sets in the evaluated settings.

This analysis complements the predictor-accuracy study. The accuracy study shows that lower-quality predictors can reduce the effectiveness of cost-aware selection, while the oracle expert-set analysis shows that replacing the converged predictor with ground-truth expert sets yields little additional change. Together, these results indicate that predictor quality matters, but the converged predictor is already sufficiently accurate for the selection rule used by EcoSpec.

C. Evaluation under Different Verification Budgets (γ)

We vary the verification budget $\gamma \in \{3, 4, 5, 6\}$ to support the common default setting used in the main experiments.

Table 11. Verification-budget sensitivity on Qwen3-235B-A22B. Each entry reports Spd / Len / Exp, corresponding to end-to-end speedup, acceptance length, and active expert count.

Dataset	Method	$\gamma = 3$ Spd / Len / Exp	$\gamma = 4$ (Default) Spd / Len / Exp	$\gamma = 5$ Spd / Len / Exp	$\gamma = 6$ Spd / Len / Exp
GSM8K	EAGLE-3	1.25 / 2.45 / 21.8	1.31 / 2.54 / 22.3	1.15 / 2.62 / 27.5	1.02 / 2.68 / 31.2
	EcoSpec	1.28 / 2.15 / 16.8	**1.39** / 2.54 / 21.0	1.25 / 2.62 / 24.5	1.09 / 2.68 / 28.2
HumanEval	EAGLE-3	1.08 / 2.25 / 20.5	1.11 / 2.35 / 26.2	1.02 / 2.41 / 30.8	0.95 / 2.45 / 33.5
	EcoSpec	1.22 / 2.05 / 17.2	**1.33** / 2.32 / 21.6	1.18 / 2.41 / 25.8	1.05 / 2.45 / 29.1
MMStar	EAGLE-3	1.30 / 2.55 / 19.8	1.35 / 2.64 / 20.5	1.22 / 2.70 / 25.2	1.08 / 2.75 / 29.5
	EcoSpec	1.41 / 2.25 / 15.1	**1.57** / 2.58 / 18.4	1.38 / 2.65 / 22.5	1.15 / 2.70 / 26.2
AIME-25	EAGLE-3	1.10 / 2.30 / 21.5	1.14 / 2.41 / 22.1	1.05 / 2.45 / 27.8	0.96 / 2.50 / 30.5
	EcoSpec	1.15 / 2.08 / 16.5	**1.23** / 2.37 / 20.9	1.12 / 2.44 / 24.2	1.02 / 2.48 / 27.8
Math500	EAGLE-3	1.02 / 1.85 / 20.5	1.05 / 1.96 / 25.4	0.98 / 2.02 / 29.5	0.90 / 2.05 / 32.8
	EcoSpec	1.08 / 1.75 / 15.9	**1.13** / 1.88 / 19.4	1.05 / 1.95 / 23.8	0.98 / 2.01 / 27.5
AMC22-24	EAGLE-3	1.08 / 2.10 / 21.2	1.12 / 2.20 / 25.4	1.04 / 2.25 / 30.1	0.95 / 2.28 / 33.2
	EcoSpec	1.19 / 1.90 / 17.5	**1.28** / 2.10 / 22.2	1.15 / 2.18 / 26.4	1.04 / 2.22 / 29.8
MTBench	EAGLE-3	1.40 / 2.65 / 21.5	1.46 / 2.78 / 24.0	1.32 / 2.85 / 28.5	1.15 / 2.90 / 32.5
	EcoSpec	1.48 / 2.40 / 16.2	**1.62** / 2.44 / 19.9	1.45 / 2.55 / 24.1	1.20 / 2.60 / 28.5
Average	EAGLE-3	1.18 / 2.31 / 21.0	1.22 / 2.41 / 23.7	1.11 / 2.47 / 28.5	1.00 / 2.52 / 31.9
	EcoSpec	1.26 / 2.08 / 16.5	**1.36** / 2.32 / 20.5	1.22 / 2.40 / 24.5	1.08 / 2.45 / 28.2

Table 12. Verification-budget sensitivity on GPT-OSS-120B.

Dataset	Method	$\gamma = 3$ Spd / Len / Exp	$\gamma = 4$ (Default) Spd / Len / Exp	$\gamma = 5$ Spd / Len / Exp	$\gamma = 6$ Spd / Len / Exp
GSM8K	EAGLE-3	1.02 / 1.45 / 11.2	1.05 / 1.56 / 11.9	1.01 / 1.62 / 13.5	0.96 / 1.65 / 15.2
	EcoSpec	1.20 / 1.30 / 8.5	1.11 / 1.52 / 10.6	1.08 / 1.60 / 11.5	1.02 / 1.65 / 13.2
HumanEval	EAGLE-3	1.02 / 1.62 / 11.0	1.05 / 1.72 / 11.8	1.00 / 1.78 / 13.2	0.94 / 1.82 / 15.0
	EcoSpec	1.15 / 1.45 / 8.8	**1.26** / 1.72 / 10.7	1.18 / 1.78 / 11.9	1.09 / 1.82 / 13.5
MMStar	EAGLE-3	1.22 / 1.80 / 11.1	1.28 / 1.91 / 11.7	1.15 / 1.98 / 13.5	1.05 / 2.05 / 15.5
	EcoSpec	1.32 / 1.62 / 8.2	**1.45** / 1.90 / 10.5	1.30 / 1.98 / 11.8	1.15 / 2.05 / 13.8
AIME-25	EAGLE-3	1.12 / 1.92 / 11.5	1.17 / 2.02 / 12.2	1.08 / 2.10 / 13.8	0.98 / 2.15 / 15.9
	EcoSpec	1.21 / 1.75 / 8.6	**1.33** / 2.02 / 10.8	1.21 / 2.10 / 12.2	1.08 / 2.15 / 14.1
Math500	EAGLE-3	1.10 / 1.92 / 11.2	1.14 / 2.02 / 12.0	1.05 / 2.08 / 13.6	0.95 / 2.12 / 15.8
	EcoSpec	1.18 / 1.72 / 8.4	**1.27** / 2.00 / 10.6	1.15 / 2.08 / 11.9	1.05 / 2.12 / 14.0
AMC22-24	EAGLE-3	1.10 / 1.92 / 11.1	1.14 / 2.02 / 11.9	1.04 / 2.10 / 13.8	0.96 / 2.15 / 15.7
	EcoSpec	1.32 / 1.75 / 8.9	**1.45** / 2.02 / 10.9	1.30 / 2.10 / 12.4	1.09 / 2.15 / 14.5
MTBench	EAGLE-3	1.12 / 1.82 / 9.2	1.18 / 1.91 / 10.3	1.08 / 1.96 / 11.5	0.98 / 2.02 / 13.2
	EcoSpec	1.20 / 1.55 / 8.1	**1.30** / 1.81 / 9.6	1.20 / 1.88 / 11.2	1.08 / 1.92 / 12.8
Average	EAGLE-3	1.10 / 1.72 / 10.9	1.14 / 1.88 / 11.6	1.06 / 1.96 / 13.3	0.97 / 2.02 / 15.1
	EcoSpec	1.23 / 1.59 / 8.5	**1.31** / 1.86 / 10.6	1.20 / 1.93 / 11.8	1.08 / 1.98 / 13.7

Tables 11 and 12 report end-to-end speedup, acceptance length, and active expert count for EAGLE-3 and EcoSpec under each budget.

We select the default verification budget according to average end-to-end speedup across the seven datasets. Under this criterion, $\gamma = 4$ gives the highest average end-to-end speedup for both EAGLE-3 and EcoSpec on Qwen3-235B-A22B and GPT-OSS-120B. Increasing γ beyond 4 further increases acceptance length in many cases, but does not improve the averaged end-to-end speedup.

The sweep also shows that EcoSpec maintains its advantage over EAGLE-3 across the tested budgets. On both target models, EcoSpec consistently achieves higher average end-to-end speedup than EAGLE-3 under $\gamma \in \{3, 4, 5, 6\}$. It also uses fewer active experts on average under the same verification budget, showing that the benefit of cost-aware draft selection is not tied to a single budget choice.

Therefore, we use $\gamma = 4$ as the default verification budget for both EAGLE-3 and EcoSpec in the main experiments.

D. Expert Activation Pattern Analysis

We analyze expert-activation patterns of the evaluated MoE backbones in Fig. 4. For each model, the left panel shows the expert-activation heatmap across layers, and the right panel shows the layer-wise expert-load distribution. This analysis characterizes the expert working sets encountered during speculative verification and provides context for why active expert count is reported alongside acceptance length.

DeepSeek-V3.1. DeepSeek-V3.1 shows a relatively diffuse activation pattern. The heatmap does not concentrate on a small subset of experts, and the layer-wise load distributions are relatively narrow across most layers. This indicates that expert usage is broadly balanced across routed experts. For speculative verification, such balanced routing still makes the expert working set an important cost component: verifying multiple draft tokens can involve many distinct experts even when no small group of experts dominates the routing pattern.

Qwen3-235B-A22B and GPT-OSS-120B. Qwen3-235B-A22B and GPT-OSS-120B show more concentrated expert-activation patterns. Their heatmaps contain clearer high-usage regions, and their layer-wise load distributions have wider ranges with more visible high-load experts. These patterns indicate stronger locality in expert usage: different candidate tokens are more likely to share parts of their expert sets, making the verified expert working set sensitive to which draft tokens are selected.

Implication for cost-aware draft selection. These observations support treating the expert working set as an explicit cost component in MoE speculative decoding. Acceptance length measures how many drafted tokens are verified successfully, but it does not describe which experts are activated during verification. Two draft sets with similar acceptance length can induce different expert working sets and therefore different verification costs. EcoSpec incorporates this cost dimension during draft selection by favoring candidates with lower predicted marginal expert cost under the same verification budget.

E. Datasets and Evaluation Details

We evaluate EcoSpec on seven benchmarks covering mathematical reasoning, code generation, dialogue, and text-only inputs derived from a vision-language benchmark. All methods are evaluated with the same prompts and decoding settings on each dataset, so the reported speedup, acceptance length, and active expert count are computed under matched input conditions.

GSM8K GSM8K (Cobbe et al., 2021) contains grade-school math word problems that require multi-step arithmetic reasoning. We use the standard 5-shot Chain-of-Thought setting, where five exemplars are prepended to the input before the test question.

AIME 2025 AIME 2025 (MAA, 2025) contains competition-level mathematical problems from the American Invitational Mathematics Examination. We use a zero-shot reasoning prompt and require the final answer to be placed in a boxed format: `{question}\n Please reason step by step, and put your final answer within \boxed{}.`

MATH500 and AMC22-24 MATH500 (Hendrycks et al., 2021; Lightman et al., 2024) is a 500-problem subset of the MATH benchmark. AMC22-24 (AI-MO) contains problems from the American Mathematics Competitions from 2022 to 2024. For both datasets, we use the following zero-shot prompt: `Problem:\n {problem}\n\n Solution:`

HumanEval HumanEval (Chen et al., 2021) contains 164 Python programming problems with function signatures, docstrings, and unit tests. We use the benchmark prompts as code-generation inputs.

MT-Bench MT-Bench (Zheng et al., 2023) is a multi-turn instruction-following benchmark.

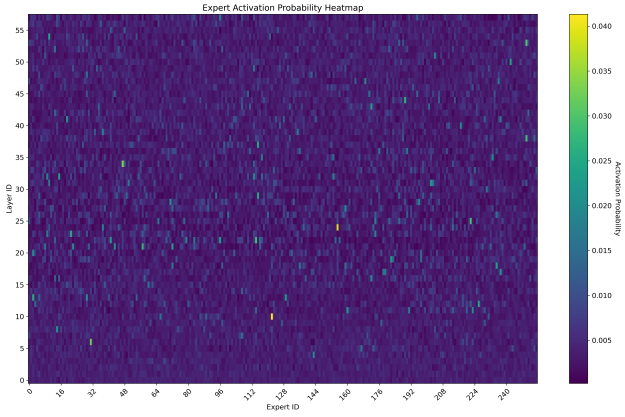
MMStar MMStar (Chen et al., 2024) is a vision-language benchmark whose original evaluation involves both image and text inputs. Since this work focuses on text-only LLM inference, we remove the image inputs and provide only the textual questions to the model. This setting is used as a text-only generation workload and is not intended to measure vision-language grounding ability.

F. Model Details

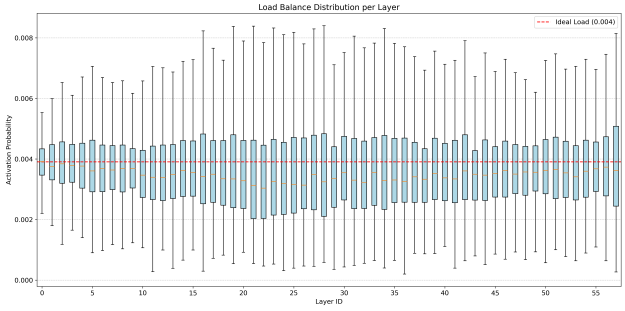
Table 13 summarizes the MoE configurations of the target models used in our experiments. We include the total and active parameter counts, the number of MoE layers, the number of experts per MoE layer, and the routing Top-k. These attributes determine the scale of expert activation during verification and are directly related to the cost-aware draft selection studied in this paper.

Table 13. Architectural summary of evaluated MoE models. Total and active parameters follow the reporting convention of the corresponding model releases. Top-k denotes the number of selected experts per token.

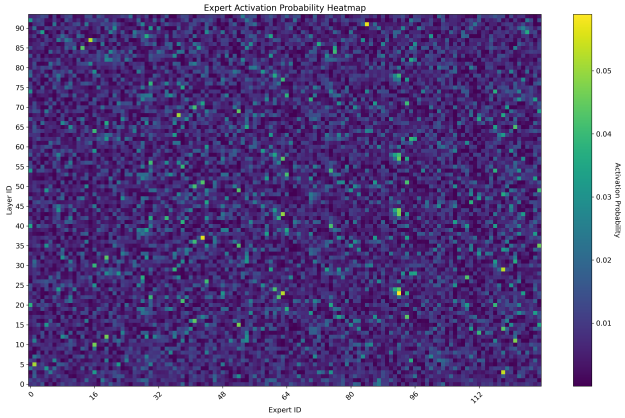
Model	DeepSeek-V3.1	Qwen3-235B-A22B	GPT-OSS-120B
Total / active parameters	671B / 37B	235B / 22B	120B / 5.1B
Transformer blocks	61	94	36
MoE / dense layers	58 / 3	94 / 0	36 / 0
Experts per MoE layer	256 routed + 1 shared	128	128
Top-k experts per token	8	8	4
Maximum context length	128K	40,960	131,072



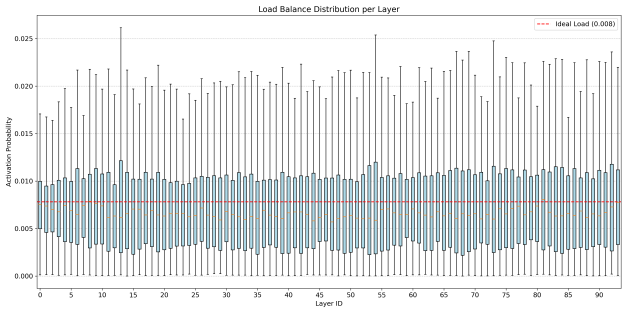
(a) **DeepSeek-V3.1**: Activation Heatmap



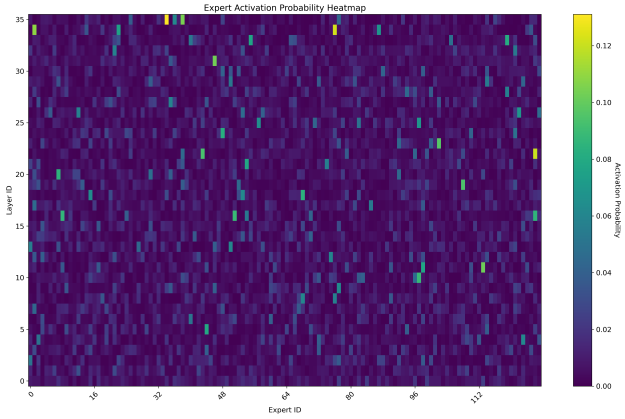
(b) **DeepSeek-V3.1**: Load Distribution



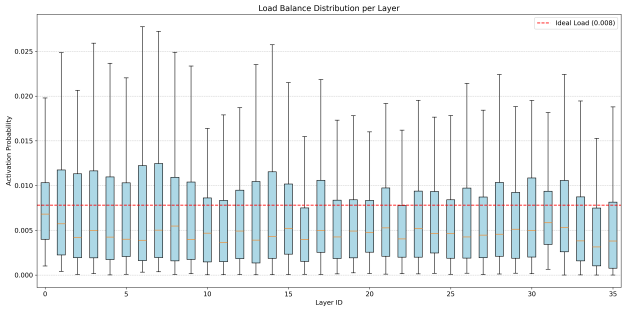
(c) **Qwen3-235B**: Activation Heatmap



(d) **Qwen3-235B**: Load Distribution



(e) **GPT-OSS-120B**: Activation Heatmap



(f) **GPT-OSS-120B**: Load Distribution

Figure 4. Expert activation patterns across evaluated MoE models. Each row corresponds to one target model. The left panel shows the expert-activation heatmap, and the right panel shows the layer-wise expert-load distribution.